J 124017

```
import java.util.concurrent.atomic.AtomicLong;
public class Monira_Thread extends Thread {
private static final AtomicLong safeStaticCounter =
new AtomicLong(0);
private static long unsafeStaticCounter = 0;
private long instancecounter = 0;
private final int increments;
private final boolean threadSafe;
public Monira_Thread(int increments, boolean threas
                     -Safe, String threadName) {

 Super(threadName);
  this.increments = increments;
  this.threadsafe = threadsafe;

 }
 @Override
Public void run() { for(
int i=0; i<increments; i++) {
if(threadSafe){
safeStaticCounter.incrementAndGet();
}else{
unsafestaticCounter++;
} instancecounter++;
 }
}
```

]124017

```
Public long getInstanceCounter () {
Return instanceCounter;
}
public static void main (String [] args)
throws InterruptedException {
 if( args.length < 3) {
System.println (" java Monira_Thread <threads> <increments,
                <true|false> ");
 return;
}
int numberofThreads = Integer.parseInt (args[0]);
if (numberof Threads <=0 || incrementsperThread <0)
{ System.outPrintln (" Threads must be>0 and
increments > 0 " ); return;
}
SafeStaticCounter.set(0);
unsafeStaticCounter =0;

Monira_Thread[ ] tasks = new Monira_Thread [numerOfTh
for(int i=0; i<numberofThreads; i++) {
 tasks[i] = new Monira_Thread (
 incrementsperThread, threadSafe,
  "Worker" + (i+1));
}
for(Monira_Thread task: tasks) {
   task.start();
  }task.join();
```

```
long nonStaticTotal = 0;
for (Main.Thread task : tasks) {
nonStaticTotal += task.getInstanceCounter(); }
long StaticCounter;
if (threadSafe) {
StaticCount = SafeStaticCounter.get();
}else{ StaticCount = UnsafeStaticCounter;
}
if (nonStaticTotal == 0) {
PercentageDifference = (StaticCounter == 0) ? 0.0 : Double.NaN;
}else{
 percentageDifference = absoluteDifference * 100.0/nonStatic
 Total;
} System.out.println ("Mode: " + (threadSafe ?
 "Thread-Safe (AtomicLong)" : "UNSYNCHRONIZED"));
System.out.println ("Number of threads: " + numberOf
 Threads);
System.out.println("Increments per thread: " + increments
 perThread);
System.println ("Excepted count: " + expectedCount);
System.out.println ("Static count: " + StaticCount);
System.out.println ("Non-static total: " + nonStaticTotal);
System.out.println ("Absolute difference: " + absoluteDiffer
```

IT24037

```
-ence);
if (Double.isNaN(percentageDifference)) {
System.out.println(
 "percentage difference: undefined");
}else{
System.out.printf(
"percentage difference: %.4f%%%n",
percentageDifference);
 }
}
}
```

1129037

Result Recording Sheet:

Threads	Expected Count	Static count	Non-Static count	Absolute difference	Difference %
1	10000	1,000	1,000	0	0.00%
2	20000	19,665	20,000	335	1.6750%
5	50000	24,374	50,000	25,626	51.2520%
10	500000	99,196	500,000	400,804	80.1608%
20	1000000	280,952	1000,000	719,048	71.9048%
50	2500000	802,817	2500,000	1,697,183	67.8873%
100	5000000	3,864,415	5000,000	1,135,585	22.711%